

“ A Smart Safety System”

Theme: WorkSafe - Building Safer Workplaces
Team ID: Safe-250345

Submitted by:
Aabiskar Regmi
Prashant Bhandari
Rajan Lamichhane
Ritushwar Neupane

Pashchimanchal Campus
Institute of Engineering
Tribhuvan University
Nepal

ABSTRACT

The Sentinel Helmet is an IoT-based safety system that monitors industrial worker's physio-logical and physical conditions in real time. The system integrates SpO_2, pulse rate, body temperature, fall detection and an SOS trigger into a standard industrial helmet using an ESP32 microcontroller and BLE communication. A neural network model which classifies health-risk while a local dashboard enables supervisors to receive alerts and respond rapidly to emergencies in workers. The prototype demonstrates reliable detection and response with less than 5 seconds of latency making it suitable for deployment in small and large-scale industries.

Keywords: Smart Helmet, IoT, ESP32, BLE, Fall Detection, SpO_2, Industrial Safety

TABLE OF CONTENT

CHAPTER 1: INTRODUCTION 3
CHAPTER 2: PROJECT OBJECTIVE 4
CHAPTER 3: LITERATURE REVIEW 5
CHAPTER 4 : METHODOLOGY 6
4.1. Hardware Implementation 6
4.1.1. Pulse/Oxygen Saturation Sensor MAX30102 6
4.1.2. Fall Detection Sensor MPU-9250 7
4.1.3. Temperature Sensor MLX90614 8
4.1.4. Micro-Controller 9
4.1.5. SOS Alert system and Buzzer 10
4.1.6. Power Management 12
4.2. Software Implementation 13
4.2.1. BLE Communications 13
4.2.2. RTOS Implementation 15
4.2.3. Data Processing Pipeline 16
4.2.3.1 BLE Listener 16
4.2.3.2 Data Processor Thread 17
4.2.4. Frontend Dashboard (HTML/CSS/JavaScript) 17
4.2.5. Backend (node.js) 17
4.2.6. Local Database using Mysql 18
4.3. ML Model and Risk Prediction 20
4.4. Mechanical Orientation and CAD 21
CHAPTER 5: COST ESTIMATIONS 24
CHAPTER 6: CONCLUSION 25
CHAPTER 7: REFERENCES 26

1. INTRODUCTION

1.1 Background

Employees in high-risk industries face intense work environments where long hours, heat, and noise can lead to physical, mental strain and health hazards. Traditional safety systems mainly monitor environmental hazards neglecting worker's physiology. The ILO reports that 80% of occupational deaths and accidents are preventable with proper strategies, including attention to workers health status, and safe handling of new technologies. Integrating these strategies with predictive health monitoring can create safer and more productive workplaces.

1.2 Problem Statement

Workers are exposed to:

1. Physical hazards
    1. Falls from height
    2. Machinery-related injuries
2. Physiological hazards
    1. Heat, stress strain
    2. Fatigue and dehydration
    3. Undetected health deterioration
3. Consequences
    1. Severe injuries and fatalities
    2. High compensation and insurance costs
    3. Delayed treatment due to unnoticed incidents in workspaces
    4. Long-term productivity decline

1.3 Scope of Work

Unlike traditional safety equipment that offers only passive protection, a Smart Helmet that provides:

1. Active health monitoring (SpO_2, pulse rate, body temperature)
2. Physical hazard detection (fall detection)
3. Emergency response for workers to call supervisor (SOS triggering)
4. Predictive health risk assessments using neural network model
5. Cost-effective solution suitable for small to large industries and scalable

2. PROJECT OBJECTIVES

1. To Develop a low-cost, power efficient IoT safety helmet which monitors vital signs like pulse rate, oxygen saturation level , fall detection and sos alert, transmits data via BLE, classifies health risks using Machine Learning Model and provides supervisors with a real-time dashboard to reduce workplace accidents.

3. LITERATURE REVIEW

[**1**] This article highlights multiple helmet systems to reduce the hazards in construction-site by IoT based helmet using gas and temperature sensors, GPS tracking, and basic alert mechanism of GSM modules. Prior works rely on simple threshold-based detection and provide limited multi-sensor integration for real time analysis.

Identified gaps:

1. Lack of integrated workers health status + physical monitoring
2. Limited predictive analytics
3. Use of high-tech cameras leading high-cost industrial solutions
4. Systems dependent on external GSM infrastructure

The Sentinel Helmet addresses these gaps by combining low-cost sensing, BLE-based data transmission, and ML-driven risk classifier in a single wearable platform. Where more features can be added in future as per the requirements and the device can be integrated in any of the respective areas.

4. Methodology

4.1. Hardware Implementation

4.1.1. Pulse/Oxygen Saturation Sensor MAX30102

The MAX30102 measures pulse rate and oxygen saturation using photoplethysmography. It emits red and infrared light to the skin and the reflected signal varies with blood volume changes during each heartbeat. The ESP32 processes these readings using filtering to remove noise caused by motion or ambient light. The sensor samples data continuously and updates the vital readings every second, allowing the system to detect abnormal drops in SpO_2 heart rate in real time. Where, we have taken the data average of the 60 seconds eliminating the noise of the heart rate and SpO_2 to gain more accuracy.

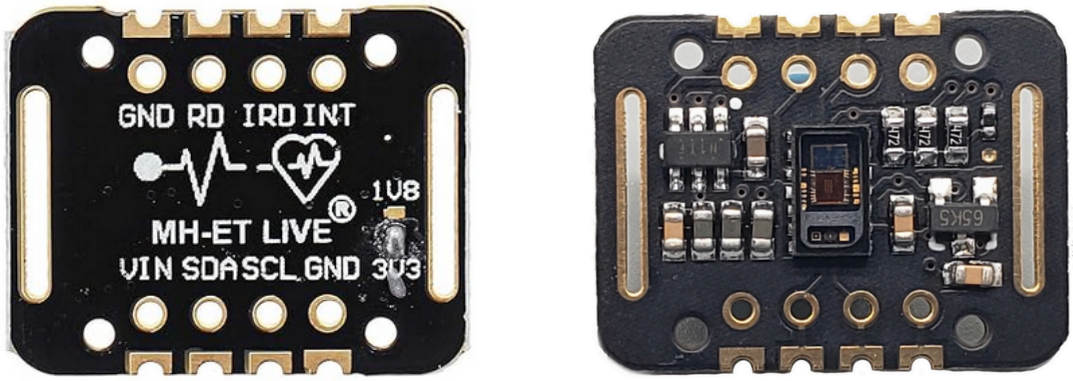


Fig 1: MAX30102 Sensor

```
long irValue = particleSensor.getIR();
bool fingerPresent = irValue > 30000;
beatDetected = checkForBeat(irValue);
if (beatDetected) {
  long delta = millis() - lastBeat;
  lastBeat = millis();

  beatsPerMinute = 60 / (delta / 1000.0);

  if (beatsPerMinute > 20 && beatsPerMinute < 255) {
    rates[rateSpot++] = (byte)beatsPerMinute; // short-term average
    rateSpot %= RATE_SIZE;
    // Short-term avg
    byte shortAvg = 0;
    for (byte i = 0; i < RATE_SIZE; i++)
      shortAvg += rates[i];
    shortAvg /= RATE_SIZE;
    hrSum += shortAvg;
    hrCount++;
  }
```

Fig 2: Detecting of the skin & Sampling Data Via Sensor

```
if (beatDetected) {
  if (beatsPerMinute > 20 && beatsPerMinute < 255) {
  }
}
// ---------- SPO2 PROCESS (100 sample buffer) ----------
static byte idx = 0;
if (particleSensor.available()) {
  redBuffer[idx] = particleSensor.getRed();
  irBuffer[idx] = particleSensor.getIR();
  particleSensor.nextSample();
  idx++;

  if (idx >= 100) {
    maxim_heart_rate_and_oxygen_saturation(
      irBuffer, 100, redBuffer,
      &spo2, &validSPO2, &calcHR, &validHR
    );

    if (validSPO2 && spo2 > 70 && spo2 < 100) {
      spo2Sum += spo2;
      spo2Count++;
    }

    idx = 0; // reset buffer index
  }
}
```

Fig 3: Sampling for SpO2

4.1.2. Fall detection Sensor MPU-9250/6500

The MPU-9250 combines a 3-axis accelerometer and gyroscope to track head movement. During normal activity, acceleration remains within a stable range but if a fall event occurs a sudden spike followed by motionless inactivity. The system identifies a fall when the acceleration exceeds approximately 2.5g and then drops below 0.3g for more than two seconds. Once this is confirmed, the ESP32 immediately triggers an alert and transmits the event to the dashboard.

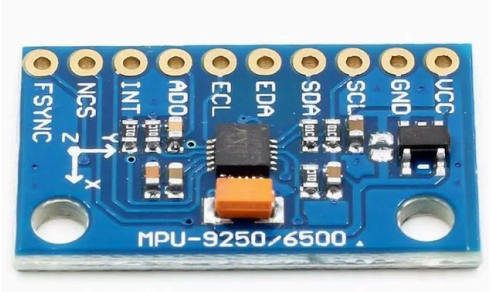


Fig 4: MPU-9250/6500

```
float g = sqrt(ax*ax + ay*ay + az*az);
// 1. Impact detection (>2.5g)
if (!impactDetected && g > 2.5) {
  impactDetected = true;
  impactTime = millis();
  // Serial.println("Impact detected...");
  mpu_msg1="Impact detected";
}
else{
  mpu_msg1="Not detected";
}
// 2. Fall confirmation
if (impactDetected) {
    if (millis() - impactTime < 2000) {
        // Serial.println("FALL DETECTED!");
        impactDetected = false;
        mpu_msg2="FALL DETECTED!";
      }
else {
      impactDetected = false;
      // Serial.println("Fall not confirmed!");
      mpu_msg2="FALL not confirmed !";

    }
  }      You, 6 days ago • Initial commit: SafeTonics Multi-Sensor Helmet …
String payload = String(heart) + "," + String(spo2) + "," + String(temp) + "," + mpu_msg1+","+mpu_msg2 ;
sensorChar->setValue(payload.c_str());
sensorChar->notify();
```

Fig 5: Testing Code for the Fall detection and message notified via BLE

4.1.3. Temperature Sensor MLX90614

The MLX90614 is a non-contact infrared thermometer that measures body temperature without touching the skin. It detects thermal radiation and converts it into a precise temperature reading with an accuracy of ±0.2°C. Because it mounts inside the helmet without requiring direct contact, it avoids hygiene issues and is unaffected by sweat. The sensor updates temperature at regular intervals and alerts the system if the value exceeds safe thresholds, such as heat stress conditions.

```
Serial.print(mlx.readAmbientTempC());
Serial.print(" °C  Object: ");
Serial.print(mlx.readObjectTempC());
```

Fig 6: Reading the ambient temperature and body temperature



Fig 7: MLX 90614 Sensor

4.1.5. Micro-Controller

The ESP32 serves as the central processing unit, collecting sensor data, executing fall-detection logic, and transmitting readings over BLE. Its dual-core architecture allows parallel handling of real-time tasks, such as sampling sensors while maintaining a stable communication link. The built-in BLE reduces external hardware requirements and keeps the overall cost and power usage low.

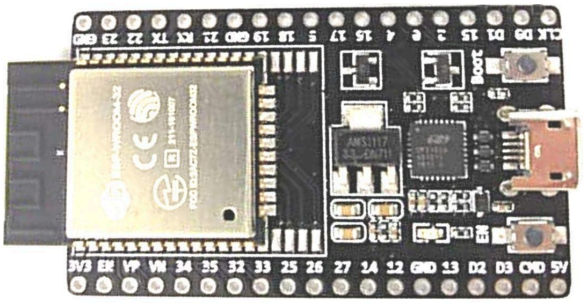


Fig 8: Esp32 Dev Module

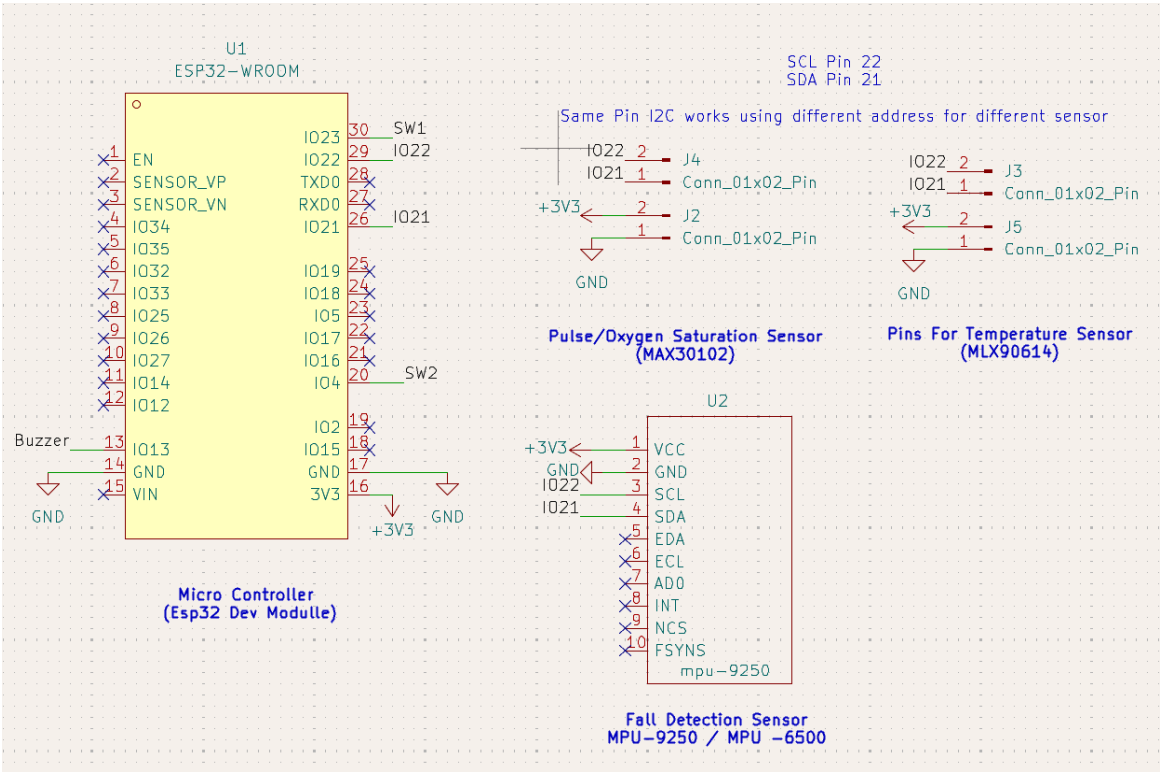


Fig 9: Schematics of the Micro Controller and Sensor

4.1.4. SOS Alert system and Buzzer

A dedicated push-button is integrated for manual emergency reporting. When pressed, the ESP32 overrides normal data transmission and prioritizes the SOS event. The alert is sent instantly to the dashboard, allowing supervisors to act even if no physiological abnormality or fall is detected. This manual trigger ensures the worker can request help during situations like entrapment or machinery malfunction.

```
84  void IRAM_ATTR button1ISR() {
85    button1Pressed = true;
86  }
87
88  void IRAM_ATTR button2ISR() {
89    button2Pressed = true;
90  }
91
92  // Buzzer Control Functions
93  void buzzerShortBeep() {
94    shortBeepActive = true;
95    shortBeepStartTime = millis();
96    digitalWrite(buzzerPin, HIGH);
97    // Serial.println("SOS Buzzer beep started");
98  }
99
100 void startBuzzerAlert() {
101   buzzerActive = true;
102 }
103
104 void stopBuzzerAlert() {
105   buzzerActive = false;
106   digitalWrite(buzzerPin, LOW);
107 }
```

Fig 10: Button as the Interrupt for Alerts

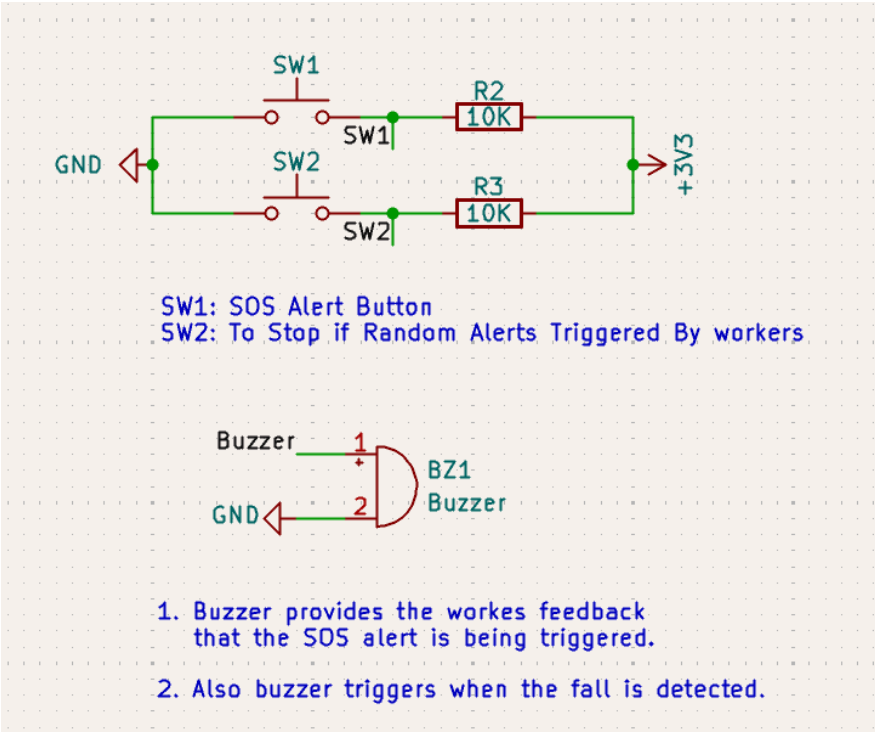
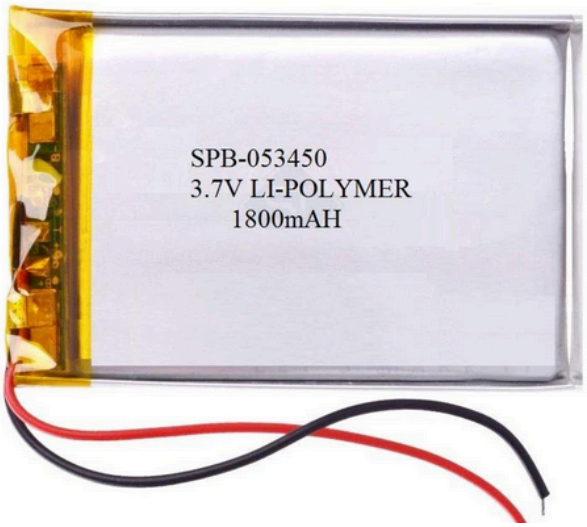


Fig 11: Schematics of the Buttons and Buzzers

4.1.4. Power Management

The system operates on a 3.7V 1800mAh Li-Po battery controlled through a TP4056 charging and protection module. The average current draw ranges from 120-150mA depending on BLE transmission frequency and sensor polling or the number of alerts given. At this consumption level, the helmet can run continuously for approximately 12–15 hours, covering a full work shift. Power distribution is regulated to prevent overheating and deep-discharge, ensuring safe wearable operation.



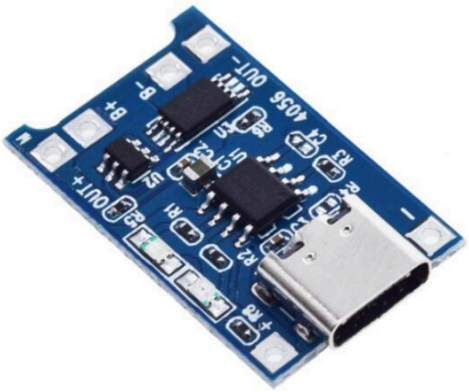


Fig 12: Li-po 3.7V Pouch Battery and Tp-4056 Charging Module

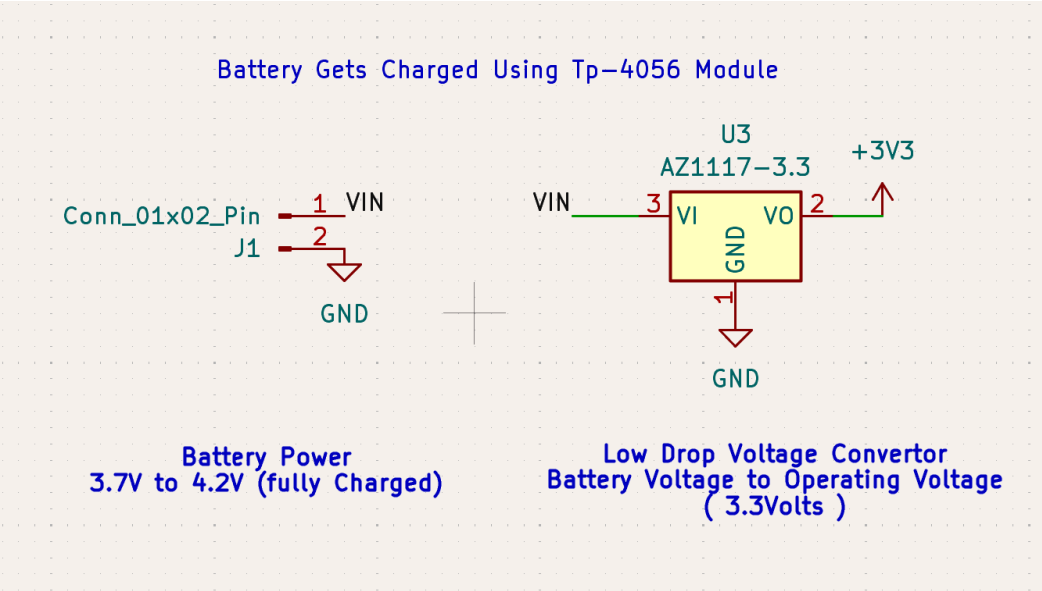


Fig 13: Schematics of the Battery and Voltage Convertor

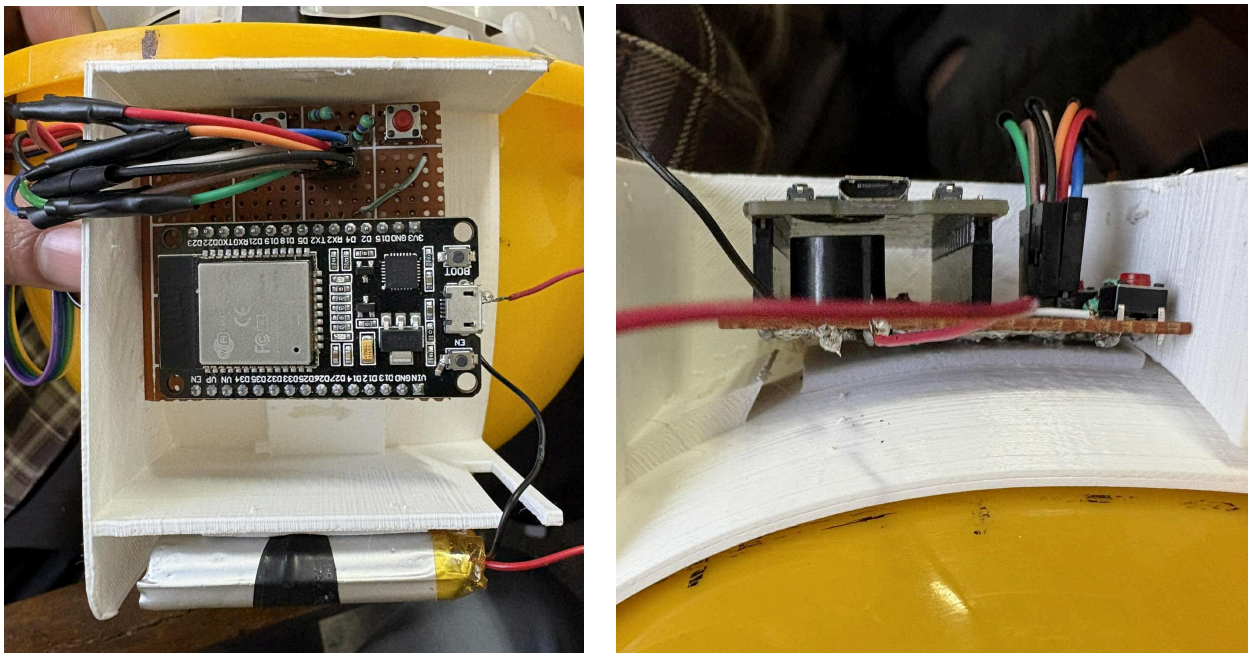


Fig 14: Top and Side View of device

4.2. Software Implementation

4.2.1. BLE Communications

Bluetooth Low Energy is used to transfer data from the helmet to a local gateway within a range of about 100 meters. BLE was selected instead of long-range technologies like LoRa

because it offers lower power consumption and is well-suited for indoor industrial environments. Data packets containing vitals and alert status are sent at regular intervals, and emergency events are pushed immediately without delay.

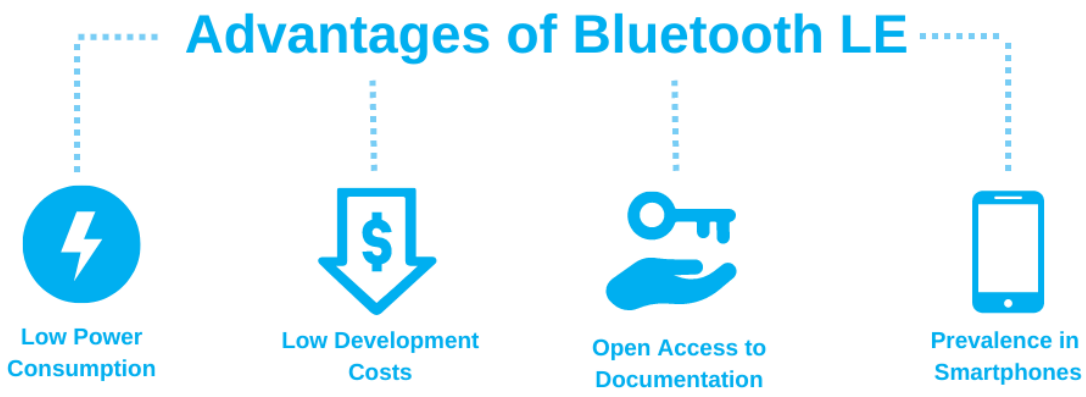


Fig 15: Advantages of Bluetooth Low Energy

Bluetooth Low Energy is a protocol. And just like any other protocol, it acts as a rulebook that devices must adhere to in order to communicate or exchange information. The Bluetooth protocol stack is broadly broken down into three primary components or subsystems i.e. application, host, and controller**.** Inside each of these blocks are distinct layers.

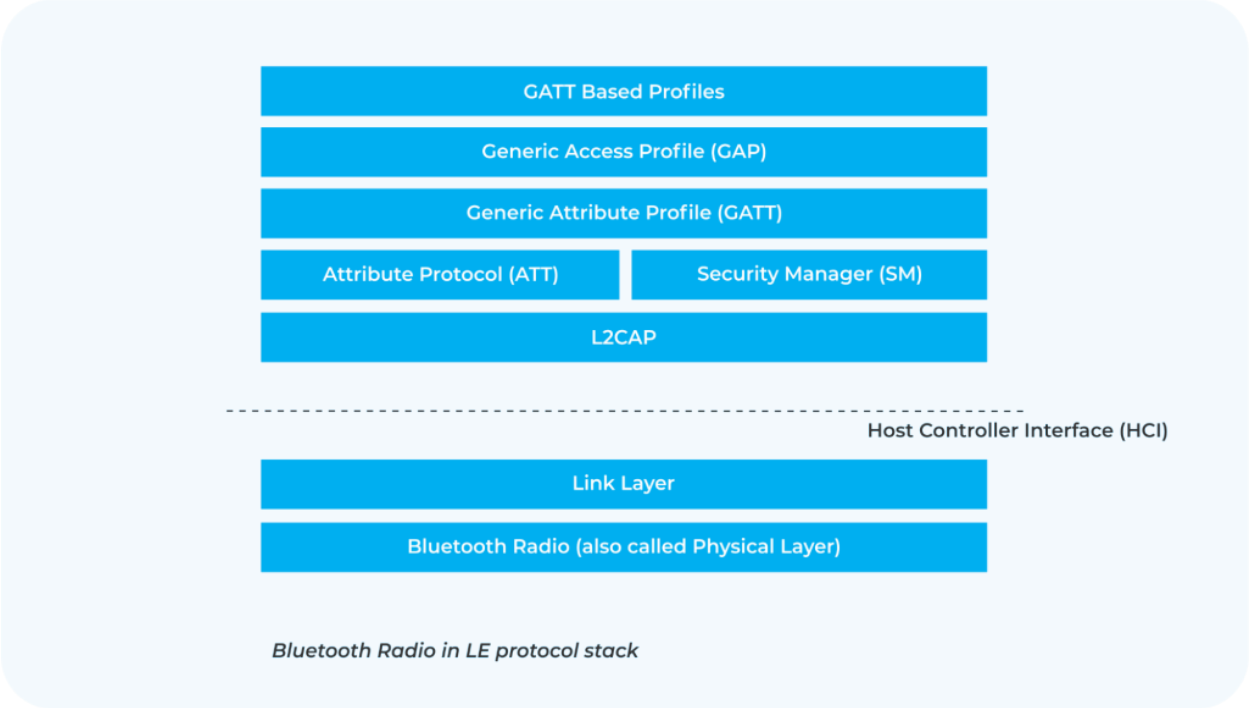


Fig 16: Bluetooth Low Energy Protocol Architecture

Note: Can use LoRa WAN for longer range, here BLE was chosen for its lower cost and lower power consumption and sufficient range for most industries which is 100M.

```
BLEDevice::init("SafeTonics-Helmet01");
pServer = BLEDevice::createServer();
pServer->setCallbacks(new MyServerCallbacks());

pService = pServer->createService("1234");

sensorChar = pService->createCharacteristic(
  "5678",
  BLECharacteristic::PROPERTY_READ | BLECharacteristic::PROPERTY_NOTIFY
);

sensorChar->addDescriptor(new BLE2902());
pService->start();

// Set initial value
sensorChar->setValue("0,0,0.0,0.0,,");

// Start advertising
BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
pAdvertising->addServiceUUID("1234");
pAdvertising->setScanResponse(false);
pAdvertising->setMinPreferred(0x0);
BLEDevice::startAdvertising();
```

Fig 17: Advertising the sensor data with unique UUIDs

4.2.2. RTOS Implementation

Real-Time Operating System (RTOS) is used to divide system functions into separate tasks running simultaneously. One task continuously reads sensor data and another manages the BLE transmission and a third handles emergency alerts. This prevents delays and ensures that high-priority events such as a fall or SOS press are processed instantly even when the system is busy. The multitasking approach improves reliability and prevents system freezing.

RTOS is an essential component for managing tasks, resources, and interrupts in real-time applications. It employs task scheduling techniques to allocate specific time periods for executing different tasks on a microcontroller.

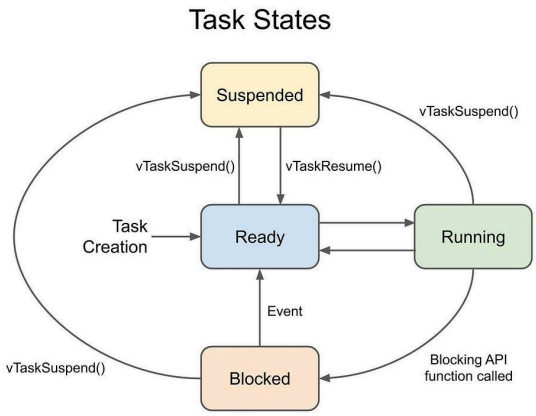


Fig 18: State representation of Task

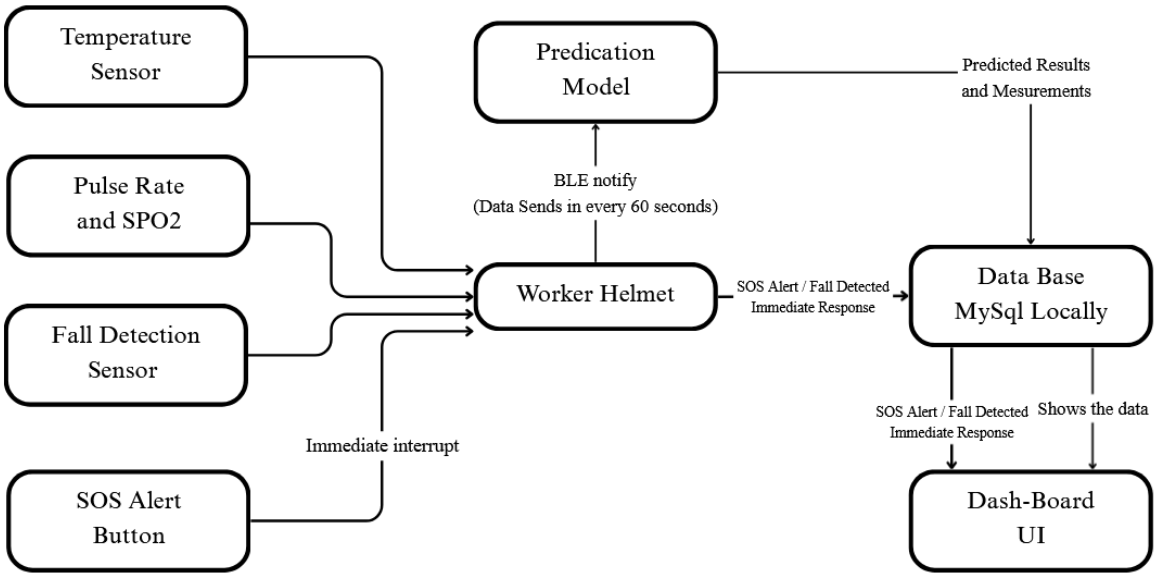


Fig 19: Flow Chart From Worker Helmet to Dash Board

4.2.3. Data Processing Pipeline

4.2.3.1 BLE Listener

Data from remote esp32 devices is received by the central device over BLE. For this, a BLE listener is implemented using asynchronous methods in python which is converted to json and pushed on the queue. Since BLE communication is an I/O-bound operation where the receiver spends most of its time waiting for notification, asynchronous methods handle it efficiently.

4.2.3.2 Data Processor Thread

The communication between the BLE listener and the data processor thread is done through a queue. The processor pull data from the queue and if emergency alerts i.e. sos alerts or fall_alerts is there then it connect to local mysql database and push the data into corresponding table else the processor fetch the other basic information from the database, predict the health risk and push the final measurements to the corresponding table.

4.2.4. Frontend (HTML/CSS/JavaScript)

The user interface for the real-time monitoring and alert visibility is built using HTML, CSS and Javascript for fast load times. CSS helps to create grid, layout, theme and responsiveness. Chart.js is used for visualizing a health history. The dashboard is connected to the backend through a websocket client that enables the constant UI updates without reload. Components are optimised for live status indicators , alert banners, search filters.

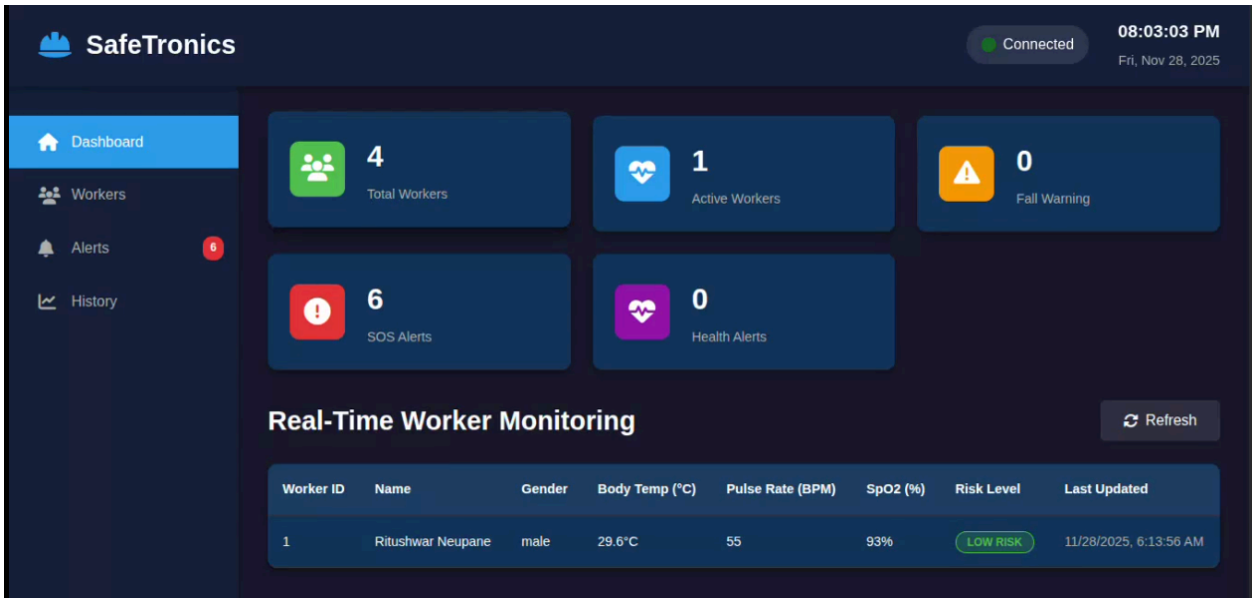


Fig 20: DashBoard UI

4.2.5. Backend (node.js)

The backend is implemented using Node.js with Express.js which provide REST API endpoints. Socket is integrated for real-time bidirectional communication. It communicates with the MySQL database through Sequalize for CRUD operations. The backend handles reconnection, websocket event routing, alert triggers and periodic dashboard refresh.

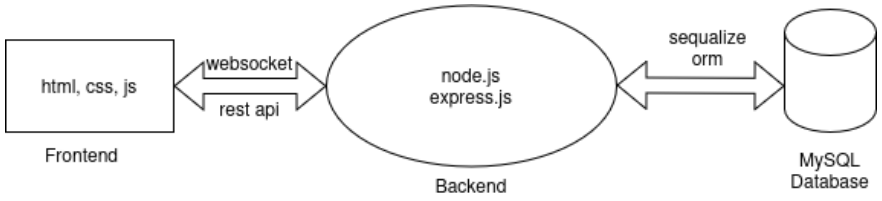
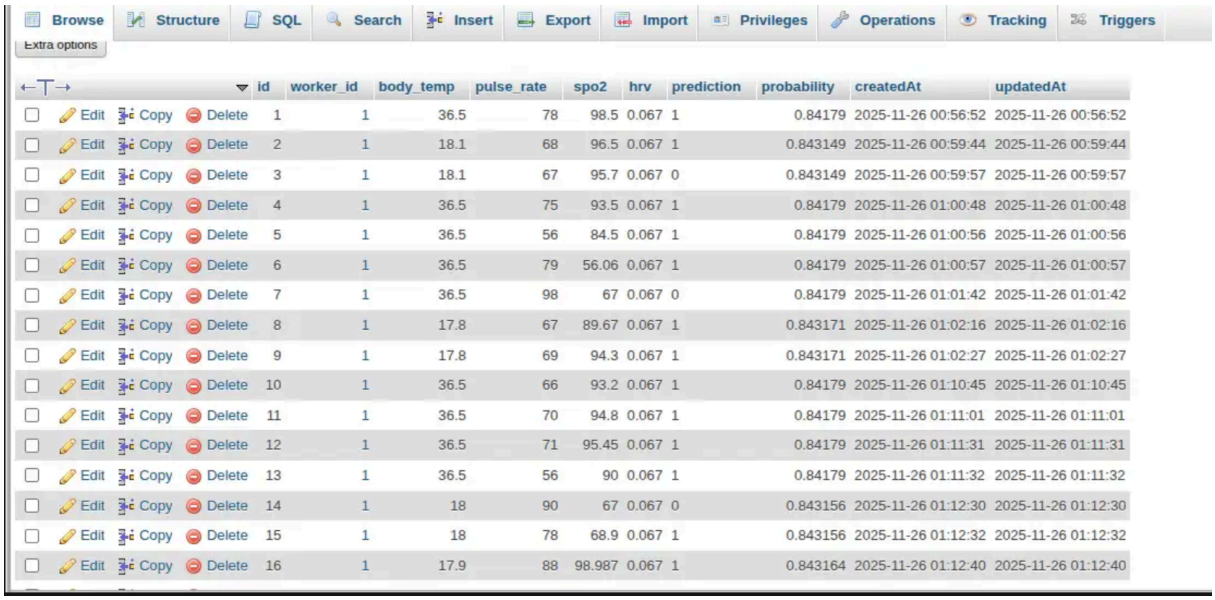


Fig 21: Safetronics Architecture

4.2.6. Local Database using Mysql

The database is created using MySQL as the primary relational system, storing structured data for workers, measurements, alerts and health history. Foreign keys maintain referential integrity between workers and their measurements, alerts and health history data.

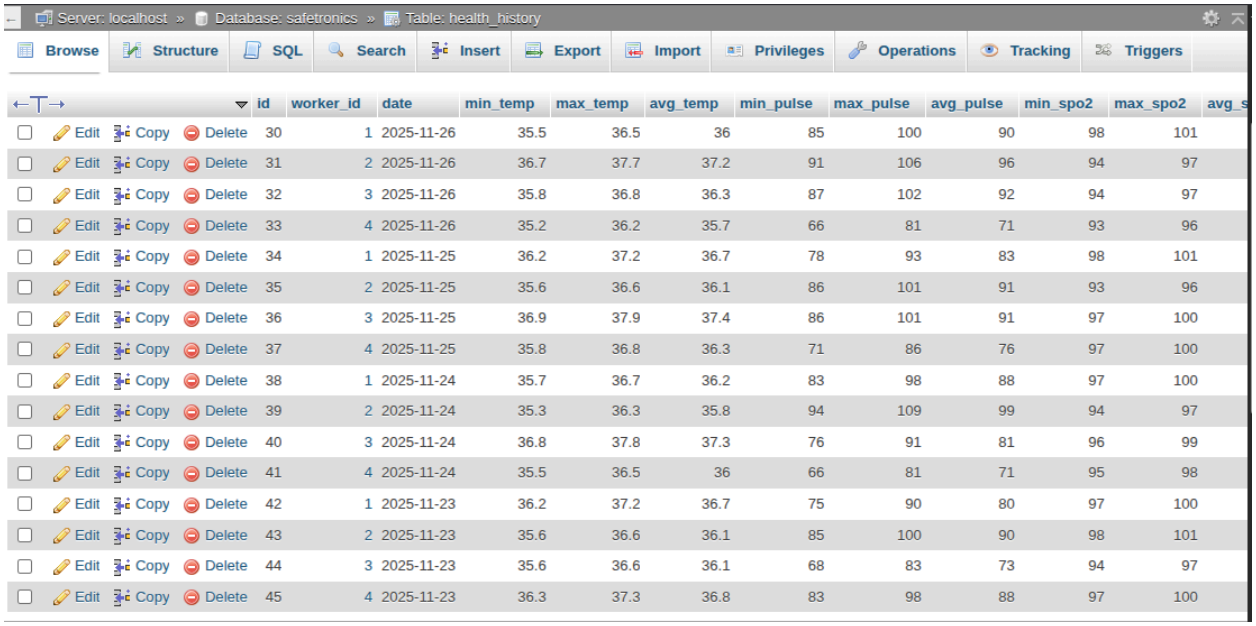


Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

Extra options

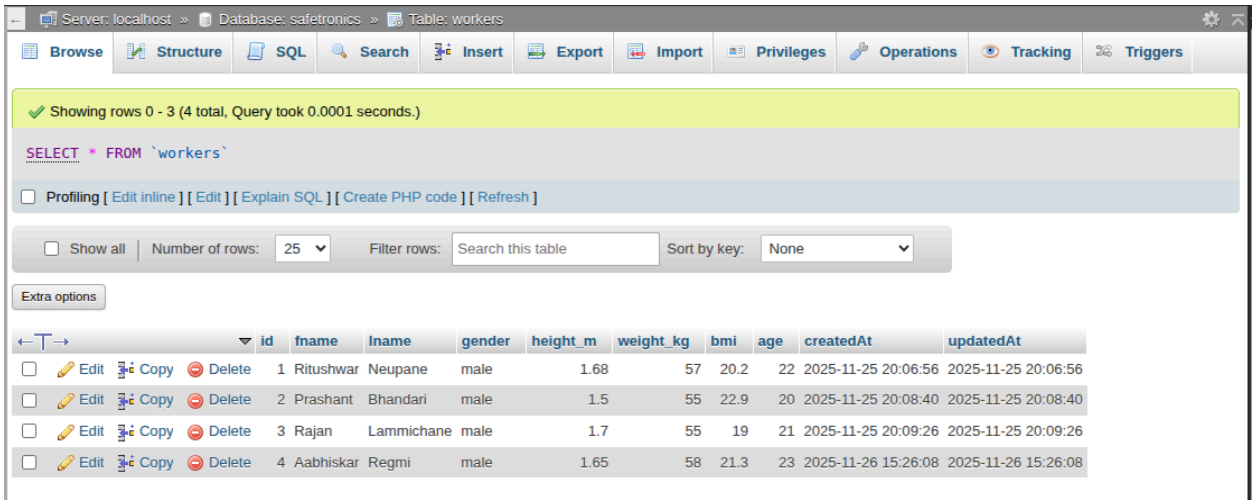
	id	worker_id	body_temp	pulse_rate	spo2	hrv	prediction	probability	createdAt	updatedAt
Edit Copy Delete	1	1	36.5	78	98.5	0.067	1	0.84179	2025-11-26 00:56:52	2025-11-26 00:56:52
Edit Copy Delete	2	1	18.1	68	96.5	0.067	1	0.843149	2025-11-26 00:59:44	2025-11-26 00:59:44
Edit Copy Delete	3	1	18.1	67	95.7	0.067	0	0.843149	2025-11-26 00:59:57	2025-11-26 00:59:57
Edit Copy Delete	4	1	36.5	75	93.5	0.067	1	0.84179	2025-11-26 01:00:48	2025-11-26 01:00:48
Edit Copy Delete	5	1	36.5	56	84.5	0.067	1	0.84179	2025-11-26 01:00:56	2025-11-26 01:00:56
Edit Copy Delete	6	1	36.5	79	56.06	0.067	1	0.84179	2025-11-26 01:00:57	2025-11-26 01:00:57
Edit Copy Delete	7	1	36.5	98	67	0.067	0	0.84179	2025-11-26 01:01:42	2025-11-26 01:01:42
Edit Copy Delete	8	1	17.8	67	89.67	0.067	1	0.843171	2025-11-26 01:02:16	2025-11-26 01:02:16
Edit Copy Delete	9	1	17.8	69	94.3	0.067	1	0.843171	2025-11-26 01:02:27	2025-11-26 01:02:27
Edit Copy Delete	10	1	36.5	66	93.2	0.067	1	0.84179	2025-11-26 01:10:45	2025-11-26 01:10:45
Edit Copy Delete	11	1	36.5	70	94.8	0.067	1	0.84179	2025-11-26 01:11:01	2025-11-26 01:11:01
Edit Copy Delete	12	1	36.5	71	95.45	0.067	1	0.84179	2025-11-26 01:11:31	2025-11-26 01:11:31
Edit Copy Delete	13	1	36.5	56	90	0.067	1	0.84179	2025-11-26 01:11:32	2025-11-26 01:11:32
Edit Copy Delete	14	1	18	90	67	0.067	0	0.843156	2025-11-26 01:12:30	2025-11-26 01:12:30
Edit Copy Delete	15	1	18	78	68.9	0.067	1	0.843156	2025-11-26 01:12:32	2025-11-26 01:12:32
Edit Copy Delete	16	1	17.9	88	98.987	0.067	1	0.843164	2025-11-26 01:12:40	2025-11-26 01:12:40

Fig 22: Locally managed Database (MySql) for measurement



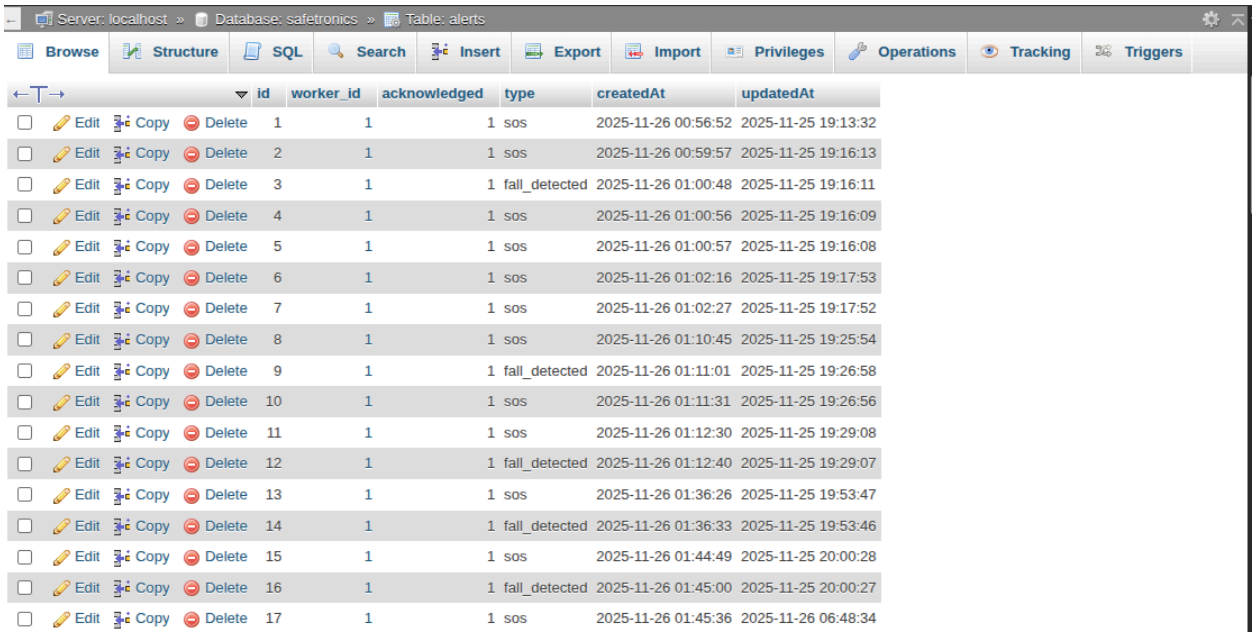
	id	worker_id	date	min_temp	max_temp	avg_temp	min_pulse	max_pulse	avg_pulse	min_spo2	max_spo2	avg_s
Edit Copy Delete	30	1	2025-11-26	35.5	36.5	36	85	100	90	98	101	
Edit Copy Delete	31	2	2025-11-26	36.7	37.7	37.2	91	106	96	94	97	
Edit Copy Delete	32	3	2025-11-26	35.8	36.8	36.3	87	102	92	94	97	
Edit Copy Delete	33	4	2025-11-26	35.2	36.2	35.7	66	81	71	93	96	
Edit Copy Delete	34	1	2025-11-25	36.2	37.2	36.7	78	93	83	98	101	
Edit Copy Delete	35	2	2025-11-25	35.6	36.6	36.1	86	101	91	93	96	
Edit Copy Delete	36	3	2025-11-25	36.9	37.9	37.4	86	101	91	97	100	
Edit Copy Delete	37	4	2025-11-25	35.8	36.8	36.3	71	86	76	97	100	
Edit Copy Delete	38	1	2025-11-24	35.7	36.7	36.2	83	98	88	97	100	
Edit Copy Delete	39	2	2025-11-24	35.3	36.3	35.8	94	109	99	94	97	
Edit Copy Delete	40	3	2025-11-24	36.8	37.8	37.3	76	91	81	96	99	
Edit Copy Delete	41	4	2025-11-24	35.5	36.5	36	66	81	71	95	98	
Edit Copy Delete	42	1	2025-11-23	36.2	37.2	36.7	75	90	80	97	100	
Edit Copy Delete	43	2	2025-11-23	35.6	36.6	36.1	85	100	90	98	101	
Edit Copy Delete	44	3	2025-11-23	35.6	36.6	36.1	68	83	73	94	97	
Edit Copy Delete	45	4	2025-11-23	36.3	37.3	36.8	83	98	88	97	100	

Fig 23: Health History of the Workers



	id	fname	lname	gender	height_m	weight_kg	bmi	age	createdAt	updatedAt
Edit Copy Delete	1	Ritushwar	Neupane	male	1.68	57	20.2	22	2025-11-25 20:06:56	2025-11-25 20:06:56
Edit Copy Delete	2	Prashant	Bhandari	male	1.5	55	22.9	20	2025-11-25 20:08:40	2025-11-25 20:08:40
Edit Copy Delete	3	Rajan	Lammichane	male	1.7	55	19	21	2025-11-25 20:09:26	2025-11-25 20:09:26
Edit Copy Delete	4	Aabhiskar	Regmi	male	1.65	58	21.3	23	2025-11-26 15:26:08	2025-11-26 15:26:08

Fig 24: Worker Details



	id	worker_id	acknowledged	type	createdAt	updatedAt
Edit Copy Delete	1	1	1	sos	2025-11-26 00:56:52	2025-11-25 19:13:32
Edit Copy Delete	2	1	1	sos	2025-11-26 00:59:57	2025-11-25 19:16:13
Edit Copy Delete	3	1	1	fall_detected	2025-11-26 01:00:48	2025-11-25 19:16:11
Edit Copy Delete	4	1	1	sos	2025-11-26 01:00:56	2025-11-25 19:16:09
Edit Copy Delete	5	1	1	sos	2025-11-26 01:00:57	2025-11-25 19:16:08
Edit Copy Delete	6	1	1	sos	2025-11-26 01:02:16	2025-11-25 19:17:53
Edit Copy Delete	7	1	1	sos	2025-11-26 01:02:27	2025-11-25 19:17:52
Edit Copy Delete	8	1	1	sos	2025-11-26 01:10:45	2025-11-25 19:25:54
Edit Copy Delete	9	1	1	fall_detected	2025-11-26 01:11:01	2025-11-25 19:26:58
Edit Copy Delete	10	1	1	sos	2025-11-26 01:11:31	2025-11-25 19:26:56
Edit Copy Delete	11	1	1	sos	2025-11-26 01:12:30	2025-11-25 19:29:08
Edit Copy Delete	12	1	1	fall_detected	2025-11-26 01:12:40	2025-11-25 19:29:07
Edit Copy Delete	13	1	1	sos	2025-11-26 01:36:26	2025-11-25 19:53:47
Edit Copy Delete	14	1	1	fall_detected	2025-11-26 01:36:33	2025-11-25 19:53:46
Edit Copy Delete	15	1	1	sos	2025-11-26 01:44:49	2025-11-25 20:00:28
Edit Copy Delete	16	1	1	fall_detected	2025-11-26 01:45:00	2025-11-25 20:00:27
Edit Copy Delete	17	1	1	sos	2025-11-26 01:45:36	2025-11-26 06:48:34

Fig 25: Alerts

4.3. ML Model and Risk Prediction

Feed Forward Neural Network(FFNN) model is trained on the human vital dataset available in kaggle which have two categories: high risk and low risk. The necessary and the impact features are taken from the dataset and train a model on a local device for 400 epochs and achieved 97.8% testing accuracy. The model consists of three hidden layers with 16,8 and 4 neurons respectively, each using ReLU activation to add non-linearity and 40% dropout to reduce overfitting. The final output layer uses a single neuron with a Sigmoid activation to produce a probability score between 0 and 1.

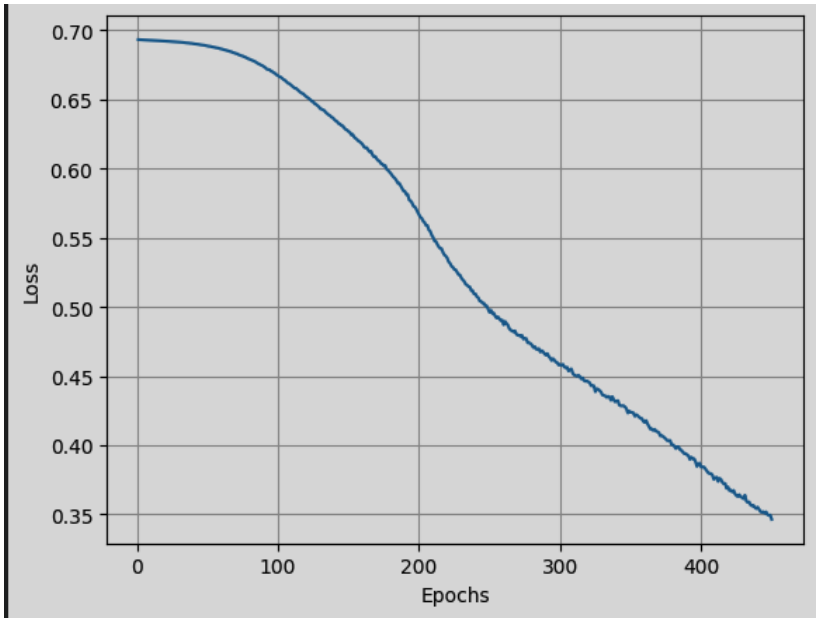


Fig 26: Epochs vs loss

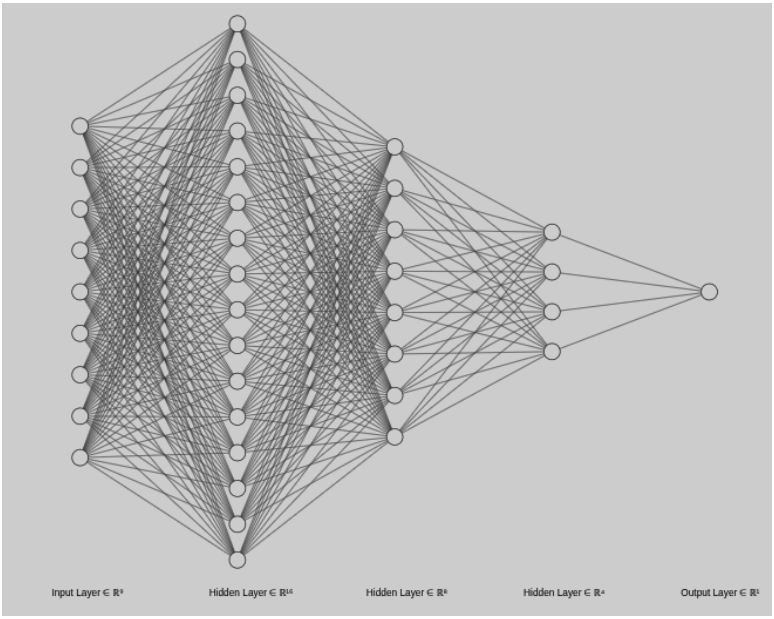


Fig 27: Architecture of FFNN

4.4. Mechanical Orientations

The entire electronic system comprising the microcontroller (ESP32), MPU, pulse oximeter and heart-rate sensor, temperature sensor, buzzer, Li-ion battery (3.7V), and TP4056 charging module is integrated into a compact module that attaches to the rear side of any standard industrial safety helmet. The module is 3D printed with PLA filament. Emphasis is given on reducing the overall size, weight, and ensuring that the helmet doesn't feel out of

ordinary. The whole housing is waterproof, with a charging port, wires and buttons extended out from it. The temperature and oximeter sensors are also housed at the appropriate places at the front side of the helmet to measure data effectively.

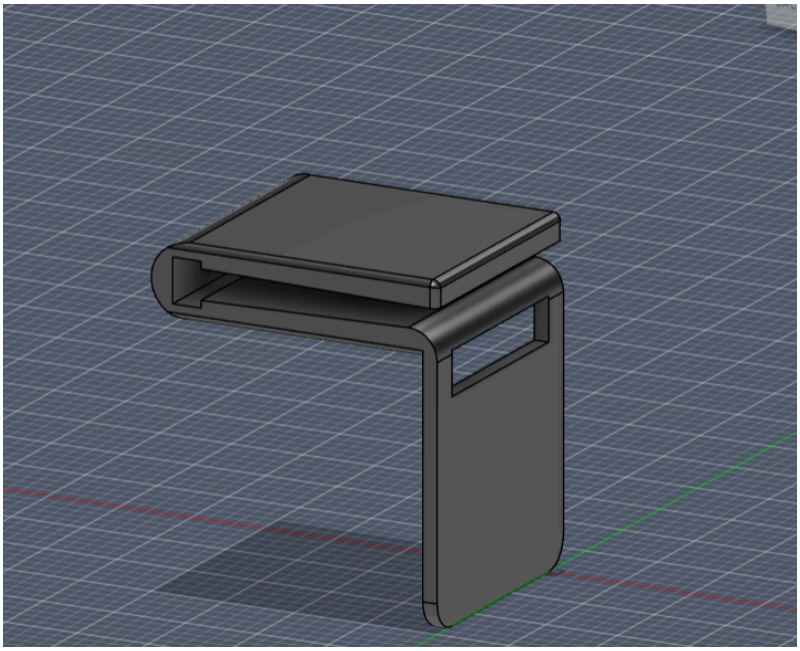


Fig 28: Temperature sensor clip

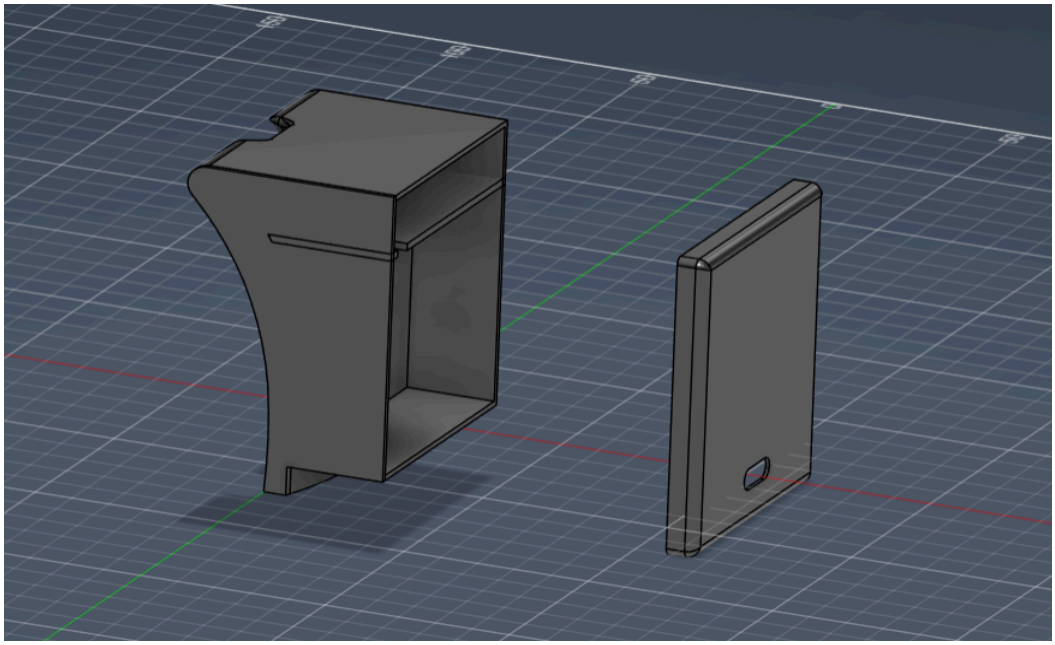


Fig 29: Isometric view of the Module casing

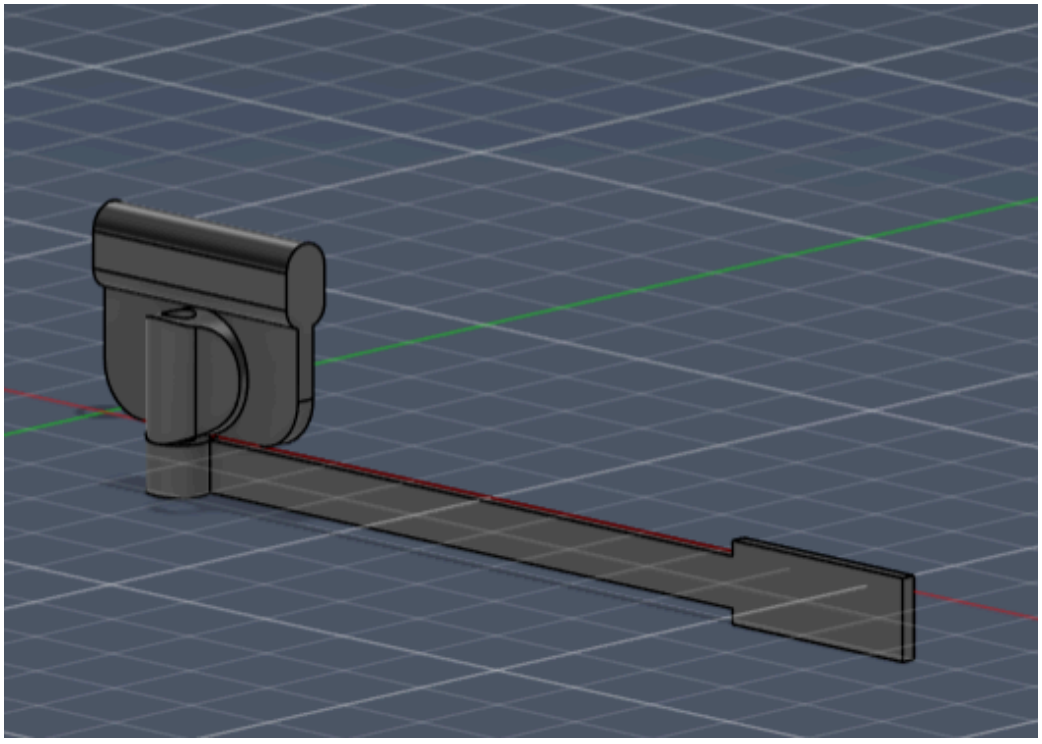


Fig 30: Pulse and oximeter sensor clip

Key design considerations:

1. Minimal added weight
2. No noticeable shift in helmet balance or comfort during prolonged use
3. Full environmental protection suitable for construction, mining, and high-temperature industrial environments

Sensor Placement for Optimal Data Accuracy

1. Non-contact infrared temperature sensor and pulse oximeter are positioned on a small 3d printed clip that sits against the worker's temple region and maintains consistent skin contact when the helmet is worn. The pod is connected to the main module via a flexible flat cable that runs around the helmet periphery.
2. The MPU is placed inside the module, alongside other electronic components to accurately detect falls, impacts, and prolonged inactivity.

This highly integrated and lightweight design provides comprehensive physiological and environmental monitoring without compromising the primary protective function of the safety helmet.

5. COST ESTIMATIONS

Components	Price(USD)
Esp-32	$10.00
MPU-9250	$10.00
Max-30102	$5.00
MLX-90614	$6.75
3.7v Pouch Battery (1800mah)	$3.00
Tp-4056 charging module	$1.00
Miscellaneous	$10.00
Total	**$45.75**

6. CONCLUSION

The Sentinel Helmet successfully integrates smart monitoring, fall detection and emergency alerting within an industrial safety helmet. The system demonstrates real-time performance, achieving response times under 5 seconds and maintaining the low cost product. The prototype proves the feasibility of affordable worker-safety wearables for industrial environments. Future development will focus on GPS tracking, cloud scalability, gas sensing, and standalone gateway deployment using Raspberry Pi.

7. REFERENCES

[1] Preeti Kuhar, Kaushal Sharma, Yaman Hooda, and Neeraj Kumar Verma.
“Internet of Things (IoT) Based Smart Helmet for Construction.” Journal of Physics: Conference Series, vol. 1950, no. 1, p. 012075, 2021. IOP Publishing.

Max-30102-High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health
Digital plug & play infrared thermometer in a TO-can
Internet of Things (IoT) based Smart Helmet for Construction
A Review on Design and Development of IoT Based Pulse Oximeter
Bluetooth Low Energy (BLE): A Complete Guide
Fundamentals of RTOS. What is a Real-time Operating System? | by Vedant A. Rokad | Medium